

Государственное бюджетное общеобразовательное учреждение города Москвы
«Бауманская инженерная школа № 1580»

**Система управления компьютерной
инфраструктурой школы
(УКК)**

Техническая документация

Архитектура · Компонентный состав · Протоколы взаимодействия
Руководство по эксплуатации · API-интерфейсы · План разработки

Версия документа: 1.0

Год: 2026

Автор: Ярош Кирилл Сергеевич(ученик 9Л класса ГБОУ №1580)

Оглавление

1	Введение и назначение системы	4
2	Базовая архитектура: фреймворк Navos	5
2.1	Компонентный состав Navos	5
2.2	Графический интерфейс Navos	5
2.3	Транспортный уровень Navos	5
2.4	Компоненты Navos, сохраняемые в УКК	7
3	Архитектура системы УКК	9
3.1	Общая схема взаимодействия	9
3.2	Центральный сервер	10
3.3	Узел-ретранслятор (Relay Node)	10
3.4	Агент УКК (переработанный Demon)	11
4	Изменения в кодовой базе Navos	12
4.1	Транспортный уровень агента	12
4.2	Профили C2 — полное удаление	12
4.3	Замена модулей задач	12
4.4	Сводная таблица изменений	14
5	Функциональные модули системы	15
5.1	Модуль скриншотов	15
5.2	Модуль управления сетевым трафиком	15
5.2.1	Белый и чёрный списки доменов	15
5.2.2	Ограничение полосы пропускания	15
5.2.3	Ограничение трафика по домену	16
5.3	Модуль мониторинга графических ресурсов	16
5.4	Модуль управления экраном	16
5.5	Модуль обнаружения P2P-трафика	16
5.6	Модуль аналитики и DNS-логирования	17
5.7	Модуль обнаружения USB-устройств	17
5.8	Watchdog и механизм обновления агента	17
6	API-интерфейсы	18

6.1	Аутентификация	18
6.2	Регистрация агента	18
6.3	Отправка команды на машины	19
6.4	Статус выполнения команды	19
6.5	Применение политики	20
6.6	Получение скриншотов	20
6.7	Панель состояния машин	21
6.8	События безопасности	21
6.9	Аналитика — сводка по зданию	22
7	Протокол взаимодействия агент — relay	23
7.1	Сообщения агент → relay	23
7.2	Сообщения relay → агент	23
7.3	Пример обмена сообщениями: экзаменационный сценарий	24
7.4	Схема состояний агента	24
8	Порядок работы с системой	26
8.1	Аутентификация операторов	26
8.2	Регламент работы в штатном режиме	26
8.3	Регламент работы в экзаменационном режиме	26
8.3.1	За 15 минут до начала экзамена	26
8.3.2	В ходе экзамена	27
8.3.3	По завершении экзамена	27
9	Аналитический модуль	28
9.1	Фазы внедрения аналитики	28
9.2	Обнаружение аномалий трафика	28
9.3	Определение схожести ответов	28
9.4	Предиктивная модель выявления учащихся в группе риска	29
10	Информационная безопасность	30
10.1	Сетевая безопасность	30
10.2	Аудит и журналирование	30
10.3	Защита данных учащихся	30
11	Система плагинов УКК	32
11.1	Проблемы существующей системы плагинов Navos	32
11.2	Преимущества подхода Adaptix	33
11.3	Архитектура системы плагинов УКК	33
11.3.1	Уровень 1: ABI-граница (C, стабильный навсегда)	33
11.3.2	Уровень 2: C++ Plugin SDK	34

11.3.3	Уровень 3: Реестр команд — ядро метапрограммирования	35
11.3.4	Уровень 4: Загрузчик плагинов с Hot-Swap	36
11.3.5	Уровень 5: Верификация возможностей при загрузке	36
11.3.6	Уровень 6: Compile-time сериализация через Boost.PFR	37
12	Инфраструктура и развёртывание	38
12.1	Серверная инфраструктура	38
12.2	Развёртывание агента	38
12.3	Централизованное обновление агентов	38

1. Введение и назначение системы

Документ является технической документацией системы УКК. Он охватывает архитектуру системы, описание компонентов, протоколы взаимодействия, API-интерфейсы, порядок эксплуатации и план разработки.

УКК представляет собой централизованную платформу дистанционного управления рабочими станциями под управлением операционной системы чМОС, развёрнутыми в образовательных учреждениях. Основными задачами системы являются:

- контроль и управление рабочими станциями учащихся в режиме реального времени;
- мониторинг и ограничение сетевого трафика;
- обеспечение информационной безопасности в ходе проведения семестровых, переводных и вступительных работ;
- сбор аналитических данных об образовательном процессе;
- автоматизация рутинных административных операций.

2. Базовая архитектура: фреймворк Navos

УКК разработан на основе C2-фреймворка Navos с сохранением транспортного уровня и архитектурных принципов диспетчеризации команд. Все наступательные модули фреймворка исключены и заменены модулями управления образовательной инфраструктурой.

2.1 Компонентный состав Navos

Исходная архитектура Navos включает следующие ключевые компоненты:

Компонент	Описание
Teamserver	Центральный сервер управления. Принимает подключения от агентов (Demon), обрабатывает задания, хранит сессии. Написан на Go.
Demon (агент)	Агентская программа, устанавливаемая на управляемый хост. Реализует цикл опроса (beacon loop), приём и исполнение задач, передачу результатов.
Профили C2	Конфигурируемые профили канала связи (malleable C2 profiles): задержки, заголовки HTTP, маскировка трафика.
Модули задач	Набор встроенных модулей: инъекция в процессы, дампы памяти, боковое перемещение по сети, кейлоггинг и прочее.
Dispatcher	Компонент маршрутизации задач от teamserver к агентам по идентификатору сессии.
Listener	Компонент приёма входящих подключений от агентов (HTTP/S, SMB, TCP).
Client (UI)	Графический интерфейс оператора на базе Qt. Отображает активные сессии, позволяет отправлять команды.

2.2 Графический интерфейс Navos

Клиент Navos реализован на Qt и представляет собой многооконный интерфейс с тёмной темой. Основные элементы интерфейса представлены ниже.

2.3 Транспортный уровень Navos

Агент Navos по умолчанию использует HTTP/S-бикон: периодически обращается к teamserver за новыми задачами с настраиваемым интервалом (sleep interval). Ответ сервера содержит зашифрованный пакет задач. Результаты исполнения передаются при следующем обращении агента.

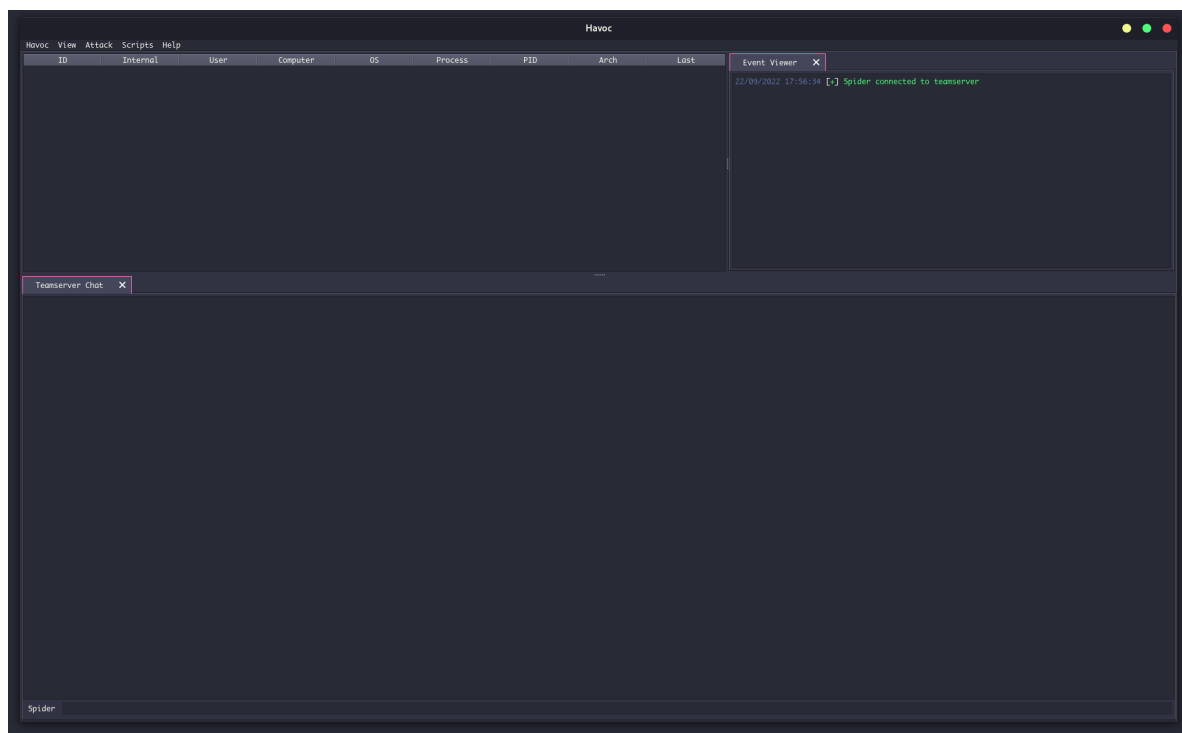


Рис. 2.1: Главное окно Havoc: таблица агентов, панель событий, чат teamserver

Модель (pull-based polling) неэффективна для управления множеством одновременно работающих агентов из-за избыточной нагрузки на сеть и высоких задержек доставки команд. В УКК транспортный уровень заменён на постоянное WebSocket-соединение через промежуточный узел-ретранслятор (relay node).

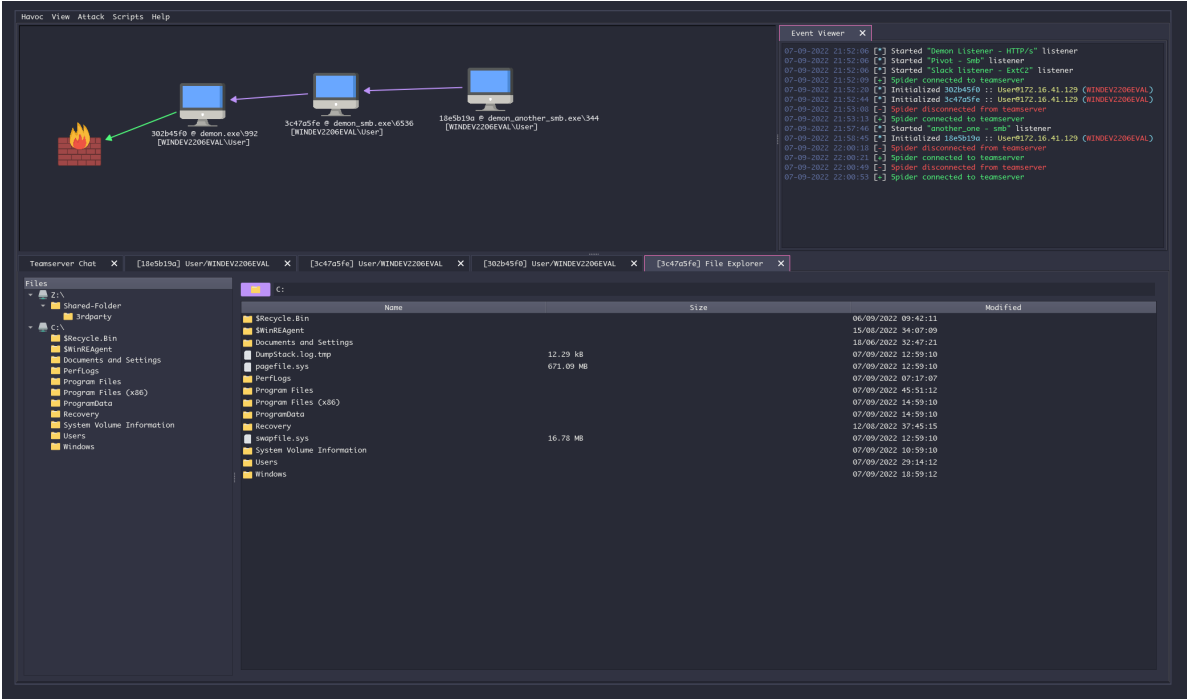


Рис. 2.2: Граф сессий: визуализация связей между агентами и teamserver

2.4 Компоненты Навос, сохраняемые в УКК

Компонент Навос	Статус в УКК
Архитектура диспетчера задач	Сохраняется. Тип задачи → обработчик.
Шифрование канала (AES/ChaCha)	Сохраняется или заменяется на mTLS.
Идентификация агента (agent UUID)	Сохраняется. Переименовывается в machine_id.
Цикл исполнения задач в агенте	Сохраняется. Модифицируется транспорт.
Профили malleable C2	Удаляются полностью. Не применимы.
Наступательные модули	Удаляются полностью. Заменяются модулями УКК.
Teamserver (Go)	Заменяется relay node + интеграция с FastAPI.
Клиентский UI (Qt)	Заменяется веб-интерфейсом УКК.

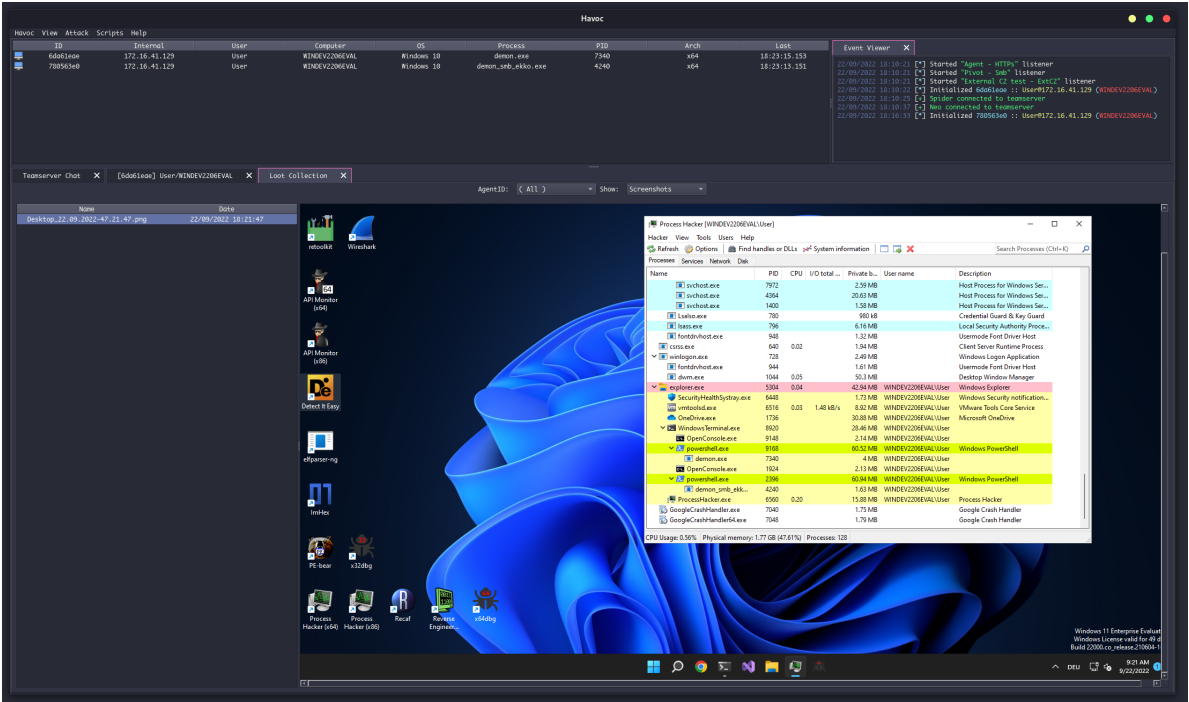


Рис. 2.3: Просмотр скриншотов и файловой системы удалённого хоста

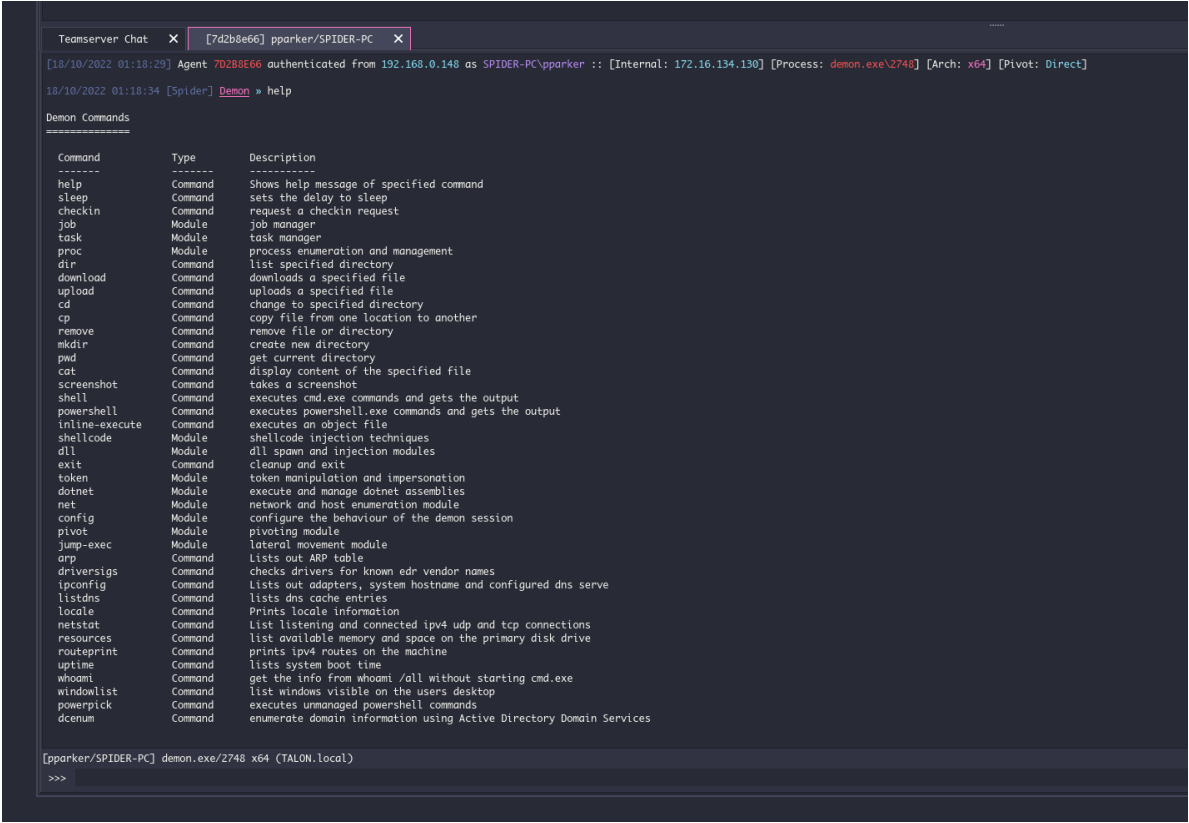


Рис. 2.4: Консоль агента: интерактивное выполнение команд

3. Архитектура системы УКК

3.1 Общая схема взаимодействия

Система построена по трёхуровневой иерархии: центральный сервер управления — узел-ретранслятор здания — агент на рабочей станции.

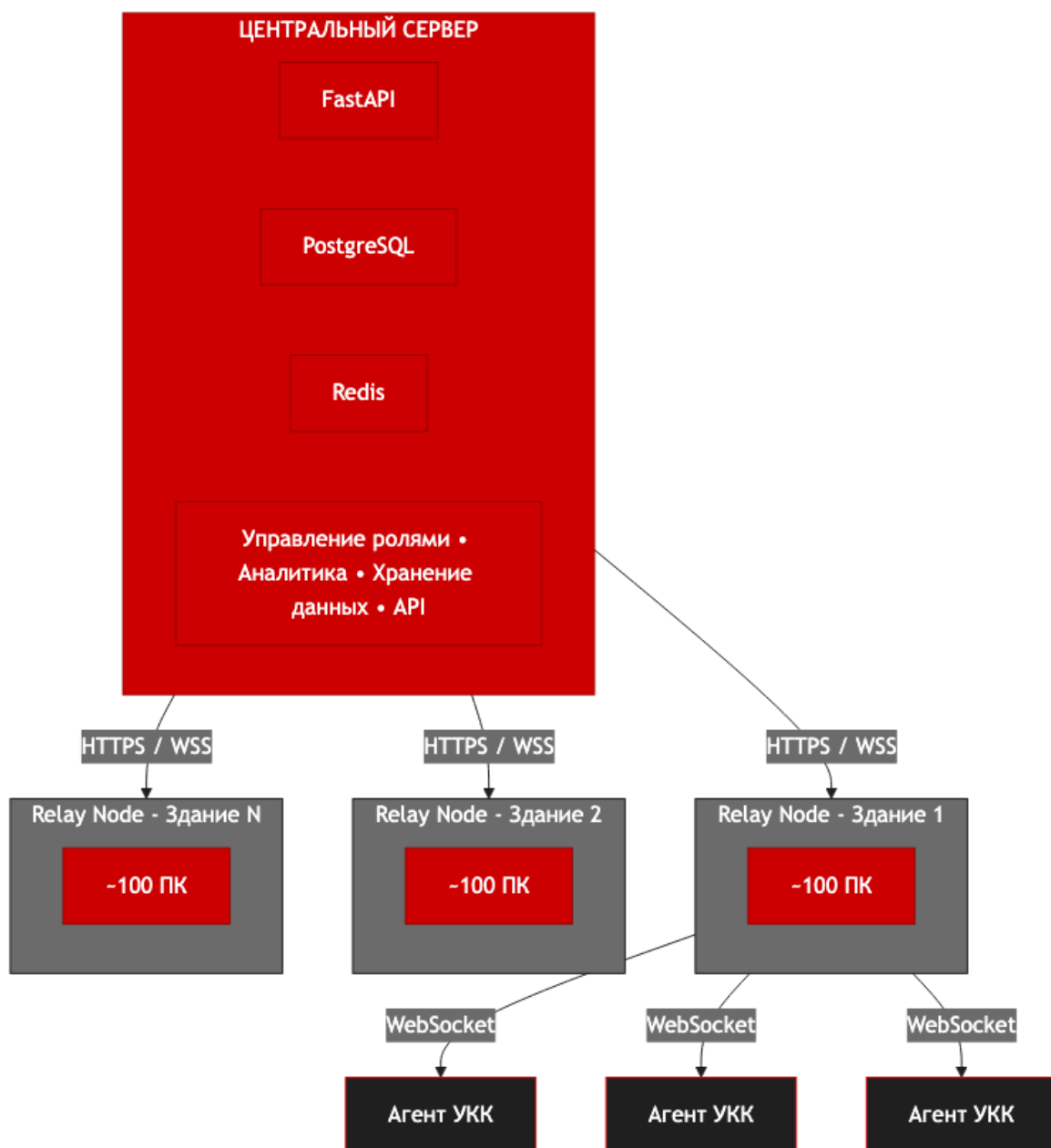


Рис. 3.1: Трёхуровневая архитектура системы УКК

3.2 Центральный сервер

Центральный сервер реализован на базе существующего проекта `mos-control-server` (FastAPI, Python 3.10+, PostgreSQL 14+, Redis 6+). Сервер не взаимодействует с агентами напрямую — все команды транслируются через `relay node` соответствующего здания.

Модуль	Функция
API Gateway	Единая точка входа для <code>relay nodes</code> , веб-интерфейса и внешних интеграций. JWT RS256-аутентификация.
Command Router	Маршрутизация команд по фильтрам: все машины / здание / аудитория / конкретная машина.
Policy Engine	Хранение и применение профилей политик (учебный, экзаменационный, свободный доступ).
Analytics Service	Агрегация метрик от <code>relay nodes</code> . Хранение в PostgreSQL. Предоставление данных аналитическому модулю.
Scheduler	Автоматическое применение профилей по расписанию учебного плана.
Audit Log	Иммутабельный журнал всех действий операторов и системных событий.

3.3 Узел-ретранслятор (Relay Node)

`Relay node` — ключевое архитектурное дополнение, отсутствующее в исходном Navos. Один `relay node` размещается в каждом здании и обслуживает все рабочие станции данного здания.

Пропускная способность канала каждого здания составляет 100 Мбит/с. `Relay node` обеспечивает локальную агрегацию трафика, что исключает прямое взаимодействие 500 агентов с центральным сервером.

Функции `relay node`:

- приём команд от центрального сервера и рассылка агентам здания (`fanout`);
- сбор и пакетная передача скриншотов с применением сжатия `zstd`;
- локальная буферизация событий при потере связи с центральным сервером;
- применение политик к агентам в `offline`-режиме на основе последней полученной конфигурации;
- агрегация метрик: трафик, активные процессы, GPU-нагрузка.

3.4 Агент УКК (переработанный Demon)

Агент устанавливается на каждую рабочую станцию. Представляет собой системный daemon (systemd unit), работающий с правами, достаточными для управления сетевыми интерфейсами, процессами и дисплейным сервером X11.

При запуске агент регистрируется на relay node здания, передавая идентификационные данные: `machine_id`, `building_id`, `room_id`, `seat_id`, версию агента, аппаратный отпечаток. Далее агент поддерживает постоянное WebSocket-соединение с relay node и исполняет поступающие задачи.

4. Изменения в кодовой базе Navoc

4.1 Транспортный уровень агента

В исходном Navoc агент реализует HTTP-бикон с настраиваемым интервалом. В УКК данный механизм заменяется постоянным WebSocket-соединением.

Причина замены: при 100 агентах на здание и интервале бикона 5 секунд нагрузка на relay составляет 20 запросов/секунду только по инициации. WebSocket исключает накладные расходы на установку соединения и обеспечивает доставку команд без задержки.

Изменения в коде агента (Demon):

- удалить функцию `NavocC2RequestHTTP()` и связанные обработчики;
- реализовать `WebSocketClient` с автоматическим переподключением (`exponential backoff`, `max 60s`);
- реализовать `heartbeat` — отправка пакета PING каждые 30 секунд;
- входящие пакеты обрабатывать в существующем диспетчере задач без изменений.

4.2 Профили C2 — полное удаление

Malleable C2-профили Navoc (файлы `.yaotl`) предназначены для маскировки C2-трафика под легитимные HTTP-запросы. В УКК данный компонент не имеет применения и подлежит полному удалению.

Удалению подлежат: парсер `.yaotl`-профилей, модуль `PrepareHTTPRequest()`, механизм трансформации заголовков, компонент обхода SSL Pinning.

4.3 Замена модулей задач

Все существующие модули задач Navoc (инъекция шеллкода, дампы LSASS, Mimikatz-интеграция и прочие) удаляются. Вместо них реализуются следующие модули:

Тип задачи (task_type)	Модуль исполнения на агенте
SCREENSHOT	Захват экрана X11 через libX11/scrot. Сжатие zstd. Отправка на relay.
NET_POLICY	Применение правил nftables/iptables. Управление dnsmasq. Контроль трафика через tc (traffic control).
PROCESS_POLICY	Мониторинг /proc. Завершение процессов по имени или PID. GPU-мониторинг через /sys/class/drm.
CURSOR_CONTROL	Управление курсором через XWarpPointer. Захват ввода через XGrabPointer.
SCREEN_CONTROL	Наложение чёрного окна (xoverlay). Управление DPMS через xset.
TRAFFIC_LIMIT	Динамические правила tc htb на основе IP-адресов, разрешённых DNS-прокси.
USB_POLICY	Применение правил udev. Блокировка/разрешение классов USB-устройств.
AGENT_UPDATE	Загрузка нового бинарного файла агента от relay. Верификация хэша SHA-256. Перезапуск.
GET_METRICS	Сбор и отправка: активный процесс, фокус окна, DNS-запросы за период, CPU/RAM/GPU.

Teamserver Havoc (Go) заменяется компонентом relay node. Relay node реализует:

- WebSocket-сервер для приёма подключений от агентов здания;
- WebSocket-клиент для подключения к центральному серверу;
- маршрутизацию команд от центрального сервера к конкретным агентам или всем агентам здания;
- агрегацию и пакетную передачу скриншотов с zstd-сжатием;
- локальное хранилище последней применённой политики (SQLite) для offline-режима;
- очередь событий для буферизации при потере связи с центральным сервером.

4.4 Сводная таблица изменений

Компонент	Действие
Beacon loop (HTTP)	Заменить на persistent WebSocket + relay
Диспетчер задач	Сохранить структуру, заменить все обработчики
Malleable C2 профили	Удалить полностью
Teamserver	Заменить на relay node + FastAPI bridge
Криптослой	Сохранить или заменить на mTLS
Логирование	Перенаправить в систему аудита УКК
Наступательные модули	Удалить; реализовать модули УКК
Client UI (Qt)	Заменить веб-интерфейсом

5. Функциональные модули системы

5.1 Модуль скриншотов

Агент захватывает скриншот рабочего стола каждые 20 секунд. Перед отправкой производится сравнение с предыдущим кадром по перцептивному хэшу (pHash). При совпадении хэшей (дельта ниже порогового значения) вместо полного скриншота отправляется уведомление об отсутствии изменений (NO_CHANGE heartbeat).

Скриншот сжимается алгоритмом zstd (уровень сжатия 3) непосредственно на агенте. Целевой размер сжатого изображения: 40–80 КБ для типичного содержимого рабочего стола.

Расчёт нагрузки на канал здания (100 агентов):

- $100 \text{ агентов} \times 60 \text{ КБ} / 20 \text{ с} = 300 \text{ КБ/с} \approx 2,4 \text{ Мбит/с}$ при полной активности;
- с учётом NO_CHANGE-фильтрации реальная нагрузка составит 10–30% от расчётной;
- выделенный лимит: 50 Мбит/с — запас достаточен.

Relay node принимает скриншоты от всех агентов здания, применяет дополнительное пакетное zstd-сжатие уровня 6 и передаёт на центральный сервер.

5.2 Модуль управления сетевым трафиком

Агент управляет сетевым трафиком рабочей станции через три механизма.

5.2.1 Белый и чёрный списки доменов

На агенте функционирует локальный DNS-прокси (dnsmasq). При получении политики NET_POLICY агент обновляет конфигурацию dnsmasq: домены из чёрного списка перенаправляются на 0.0.0.0, домены белого списка разрешаются в штатном режиме. При режиме «только белый список» все домены, не входящие в список, блокируются.

5.2.2 Ограничение полосы пропускания

Throttling реализован через подсистему traffic control Linux (tc). Для ограничения скорости используется дисциплина tbf (token bucket filter) или htb (hierarchical token bucket). Пример применяемых команд:

```
# Ограничение исходящего трафика до 512 Кбитс/
tc qdisc add dev eth0 root tbf rate 512kbit burst 32kbit latency 400ms

# Удаление ограничения
tc qdisc del dev eth0 root
```


5.2.3 Ограничение трафика по домену

DNS-прокси перехватывает ответы, извлекает разрешённые IP-адреса и динамически создаёт правила `tc` `htb` для конкретных IP-префиксов. При изменении IP-адресов домена правила обновляются автоматически. Используется `nftables`-набор (`nft set`) для хранения IP-адресов, подлежащих ограничению.

5.3 Модуль мониторинга графических ресурсов

Агент периодически опрашивает список запущенных процессов через `/proc/[pid]/comm` и `/proc/[pid]/cmdline`. Сравнение производится со списком запрещённых приложений из активной политики (игровые клиенты, медиаплееры, торрент-клиенты и прочее).

Дополнительно производится мониторинг загрузки GPU. На системах с GPU NVIDIA используется `nvidia-smi`, на системах с GPU AMD — `radeontop` или `/sys/class/drm/card*/device`. При превышении порогового значения загрузки GPU (по умолчанию 40% в течение 30 секунд) агент фиксирует событие и выполняет действие согласно политике: предупреждение оператора или завершение процесса.

Завершение процесса: `SIGTERM` → ожидание 3 секунды → `SIGKILL`. Результат (PID, имя процесса, причина) передаётся на `relay node`.

5.4 Модуль управления экраном

Модуль реализует два режима отключения экрана:

- **Программное наложение (рекомендуется):** агент запускает дочерний процесс, создающий полноэкранное окно без декораций с чёрным фоном поверх всех окон (`always-on-top`). Окно не может быть закрыто пользователем. При получении команды `SCREEN_RESTORE` дочерний процесс завершается.
- **Аппаратное отключение:** вызов `xset dpms force off` или запись значения 0 в `/sys/class/backlight/[interface]/brightness`.

Преимущество программного наложения: устойчивость к краш агента (дочерний процесс продолжает работу независимо), возможность мгновенного снятия блокировки без физического взаимодействия с оборудованием.

5.5 Модуль обнаружения P2P-трафика

Агент периодически (каждые 60 секунд) опрашивает таблицу активных соединений через `ss -tnp` и `arp-scan` локальной подсети. Соединения между машинами одного здания (принадлежащими к одной /24-подсети здания) фиксируются как события `P2P_DETECTED`.

`Relay node` агрегирует события от всех агентов здания. При обнаружении взаимных соединений между двумя машинами формируется предупреждение высокого приоритета, направляемое на центральный сервер.

Данная функция активируется только в экзаменационном профиле политики.

5.6 Модуль аналитики и DNS-логирования

Локальный DNS-прокси агента фиксирует каждый DNS-запрос: доменное имя, временная метка, PID инициировавшего запроса процесса. Сырые логи хранятся локально не более 24 часов. Каждые 5 минут агент формирует агрегированный отчёт: топ-20 запрошенных доменов за период, общее число уникальных доменов, число заблокированных запросов.

Агрегированные отчёты передаются на relay node, который накапливает данные по зданию и передаёт на центральный сервер. Сырые DNS-логи с агентов не передаются на центральный сервер и не хранятся за пределами здания.

Аналогично фиксируется фокус активного окна (имя приложения, заголовок окна без содержимого) каждые 30 секунд. Формируется timeline сессии: приложение — длительность фокуса.

5.7 Модуль обнаружения USB-устройств

Агент подписывается на события udev через libudev. При подключении USB-устройства фиксируется событие: временная метка, VID/PID устройства, класс устройства (Mass Storage, HID, CDC и прочее), machine_id.

При активированной политике блокировки USB агент применяет правило udev, запрещающее авторизацию устройств указанного класса:

```
SUBSYSTEM=="usb", ATTR{bDeviceClass}=="08", ATTR{authorized}="0"
# 08 = Mass Storage
# Применяется: udevadm control --reload-rules && udevadm trigger
```

Событие подключения USB передаётся на центральный сервер в режиме реального времени вне зависимости от применённой политики.

5.8 Watchdog и механизм обновления агента

Агент сопровождается отдельным процессом watchdog (второй systemd unit). Watchdog выполняет следующие функции:

- проверка хэша SHA-256 бинарного файла агента каждые 60 секунд;
- перезапуск агента при его завершении (в том числе принудительном);
- отправка события TAMPER_DETECTED на relay node при несоответствии хэша;
- загрузка эталонного бинарного файла с relay node и его применение при обнаружении модификации.

Обновление агента инициируется командой AGENT_UPDATE с центрального сервера. Relay node раздаёт новый бинарный файл агентам здания. Агент верифицирует хэш, завершает основной процесс, watchdog запускает обновлённую версию. Статус обновления каждой машины отображается в панели управления.

6. API-интерфейсы

Все API-эндпоинты доступны по базовому пути `/api/v1/`. Аутентификация осуществляется через JWT Bearer-токен (RS256). Токен передаётся в заголовке `Authorization: Bearer <token>`.

6.1 Аутентификация

```
POST /api/v1/auth/login
Content-Type: application/json

{
  "username":      "string",
  "password":      "string",
  "totp_code":     "string",    // значный6- TOTPкод-
  "usb_signature": "string"     // base64подпись- от USBносителя-
}

// Ответ 200:
{
  "access_token": "string",    // JWT, TTL 15 минут
  "refresh_token": "string",   // TTL 8 часов
  "role":         "string",
  "building_id":  "string | null"
}
```

6.2 Регистрация агента

```
POST /api/v1/agents/register
// Аутентификация по pre-shared key здания

{
  "machine_id":      "uuid",
  "building_id":     "string",
  "room_id":         "string",
  "seat_id":         "string",
  "agent_version":   "string",
  "hw_fingerprint":  "string",  // SHA-256(CPU + MB + MAC)
  "os_version":      "string"
}
```

```
// Ответ 200:
{
  "session_token": "string", // токен сессии агента
  "relay_ws_url": "string", // адрес WebSocket relay node
  "initial_policy": { ... } // текущая активная политика
}
```

6.3 Отправка команды на машины

```
POST /api/v1/commands/send
Authorization: Bearer <token>

{
  "target": {
    "type": "all" | "building" | "room" | "machine",
    "id": "string | null" // building_id, room_id или machine_id
  },
  "task_type": "string", // см. таблицу типов задач
  "payload": { ... }, // параметры задачи
  "priority": "normal" | "high"
}

// Ответ 202:
{
  "command_id": "uuid",
  "target_count": 42, // число машинполучателей-
  "status": "dispatched"
}
```

6.4 Статус выполнения команды

```
GET /api/v1/commands/{command_id}/status
Authorization: Bearer <token>

// Ответ 200:
{
  "command_id": "uuid",
  "total": 100,
  "acknowledged": 98,
  "failed": 1,
  "pending": 1,
  "results": [
```

```
{
  "machine_id": "uuid",
  "status":      "ok" | "error" | "timeout",
  "error":       "string | null"
}
]
```

6.5 Применение политики

```
POST /api/v1/policies/apply
Authorization: Bearer <token>

{
  "policy_id":      "uuid", // ID predetermined policy
  "target":         { ... }, // analogically /commands/send
  "override_schedule": false // ignore schedule
}

// Предопределённые профили политик:
// "exam"      -- экзаменационный: интернет отключён, USB заблокированы,
//              скриншоты каждые с20, P2Pмониторинг- активен
// "lesson"    -- учебный: белый список сайтов, USB по умолчанию
// "free"      -- свободный доступ: без ограничений
// "lockdown"  -- административный: экран заблокирован, ввод заблокирован
```

6.6 Получение скриншотов

```
GET /api/v1/screenshots
?building_id=string
&room_id=string
&machine_id=string
&limit=50
Authorization: Bearer <token>

// Ответ 200:
{
  "screenshots": [
    {
      "machine_id": "uuid",
      "timestamp":  "ISO8601",
      "image_url":  "string", // presigned URL или base64
    }
  ]
}
```

```
        "changed":    true        // false = NO_CHANGE кадр
    }
]
}
```

6.7 Панель состояния машин

```
GET /api/v1/machines
  ?building_id=string
  &room_id=string
  &status=online
Authorization: Bearer <token>

// Ответ 200:
{
  "machines": [
    {
      "machine_id":    "uuid",
      "building_id":   "string",
      "room_id":       "string",
      "seat_id":       "string",
      "status":        "online" | "offline" | "error",
      "agent_version": "string",
      "last_seen":     "ISO8601",
      "active_policy": "string",
      "cpu_pct":       12,
      "ram_pct":       45,
      "gpu_pct":       0,
      "disk_free_gb":  18.4
    }
  ]
}
```

6.8 События безопасности

```
GET /api/v1/events
  ?severity=high
  &building_id=string
  &from=ISO8601
  &to=ISO8601
Authorization: Bearer <token>
```

```
// Ответ 200:
{
  "events": [
    {
      "event_id": "uuid",
      "machine_id": "uuid",
      "event_type": "USB_CONNECTED" | "P2P_DETECTED"
                    | "TAMPER_DETECTED" | "PROCESS_KILLED"
                    | "AGENT_OFFLINE",
      "severity": "low" | "medium" | "high" | "critical",
      "timestamp": "ISO8601",
      "details": { ... }
    }
  ]
}
```

6.9 Аналитика — сводка по зданию

```
GET /api/v1/analytics/building/{building_id}/summary
  ?date=YYYY-MM-DD
Authorization: Bearer <token>

// Ответ 200:
{
  "building_id": "string",
  "date": "YYYY-MM-DD",
  "machines_total": 100,
  "machines_online_avg": 94.2,
  "top_domains": [
    { "domain": "string", "query_count": 1420 }
  ],
  "blocked_domain_count": 342,
  "usb_events": 3,
  "p2p_events": 0,
  "tamper_events": 0,
  "app_focus_summary": [
    { "app": "string", "total_minutes": 180 }
  ]
}
```

7. Протокол взаимодействия агент — relay

Взаимодействие агента с relay node осуществляется по протоколу WebSocket (`ws://` внутри здания, `wss://` при передаче данных relay → центральный сервер). Сообщения кодируются в формате JSON. Все сообщения содержат поля `type`, `message_id` (UUID) и `timestamp` (ISO 8601).

7.1 Сообщения агент → relay

Тип сообщения	Содержание и назначение
REGISTER	Регистрация агента: <code>machine_id</code> , <code>building_id</code> , <code>room_id</code> , <code>seat_id</code> , <code>agent_version</code> , <code>hw_fingerprint</code> .
HEARTBEAT	Периодическое подтверждение активности. Содержит: <code>cpu_pct</code> , <code>ram_pct</code> , <code>gpu_pct</code> , <code>disk_free_gb</code> , <code>active_policy</code> .
SCREENSHOT	Скриншот в формате base64 (zstd-сжатие). Поля: <code>changed</code> (bool), <code>image_data</code> (string null).
TASK_RESULT	Результат выполнения задачи: <code>task_id</code> , <code>status</code> (ok error), <code>output</code> (string), <code>error_code</code> (int null).
EVENT	Событие безопасности или мониторинга: <code>event_type</code> , <code>severity</code> , <code>details</code> .
METRICS	Агрегированные метрики за период: <code>dns_summary</code> , <code>app_focus_timeline</code> , <code>process_list</code> .

7.2 Сообщения relay → агент

Тип сообщения	Содержание и назначение
TASK	Задача для исполнения: <code>task_id</code> (UUID), <code>task_type</code> , <code>payload</code> , <code>priority</code> , <code>deadline</code> (ISO8601).
POLICY_UPDATE	Новая политика в полном объёме: <code>policy_id</code> , <code>rules</code> (объект с параметрами всех модулей).
PING	Проверка соединения. Агент отвечает PONG.
AGENT_UPDATE	Команда обновления: <code>new_version</code> , <code>download_url</code> , <code>sha256_hash</code> . Агент загружает файл с relay.

7.3 Пример обмена сообщениями: экзаменационный сценарий

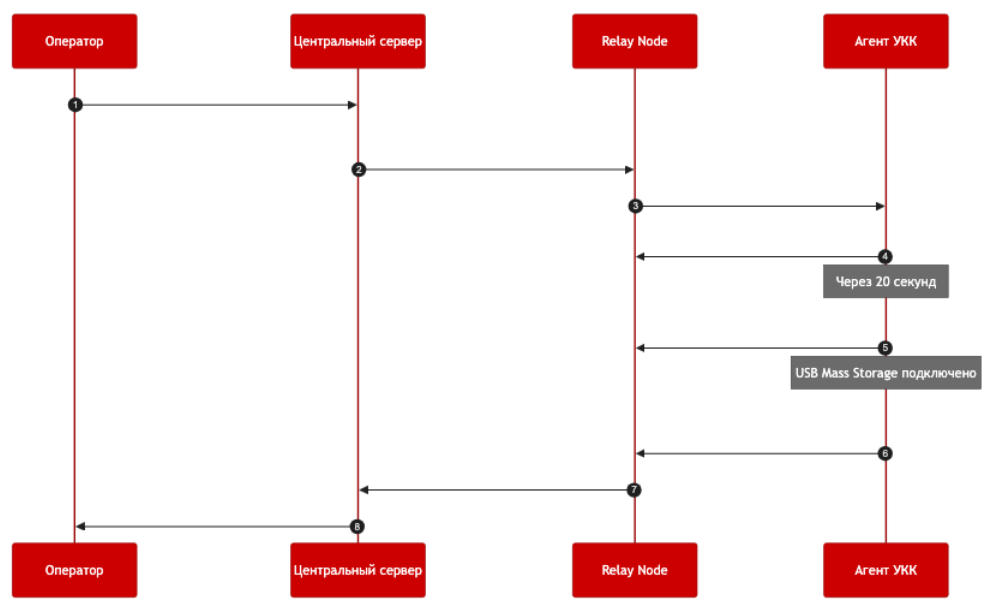
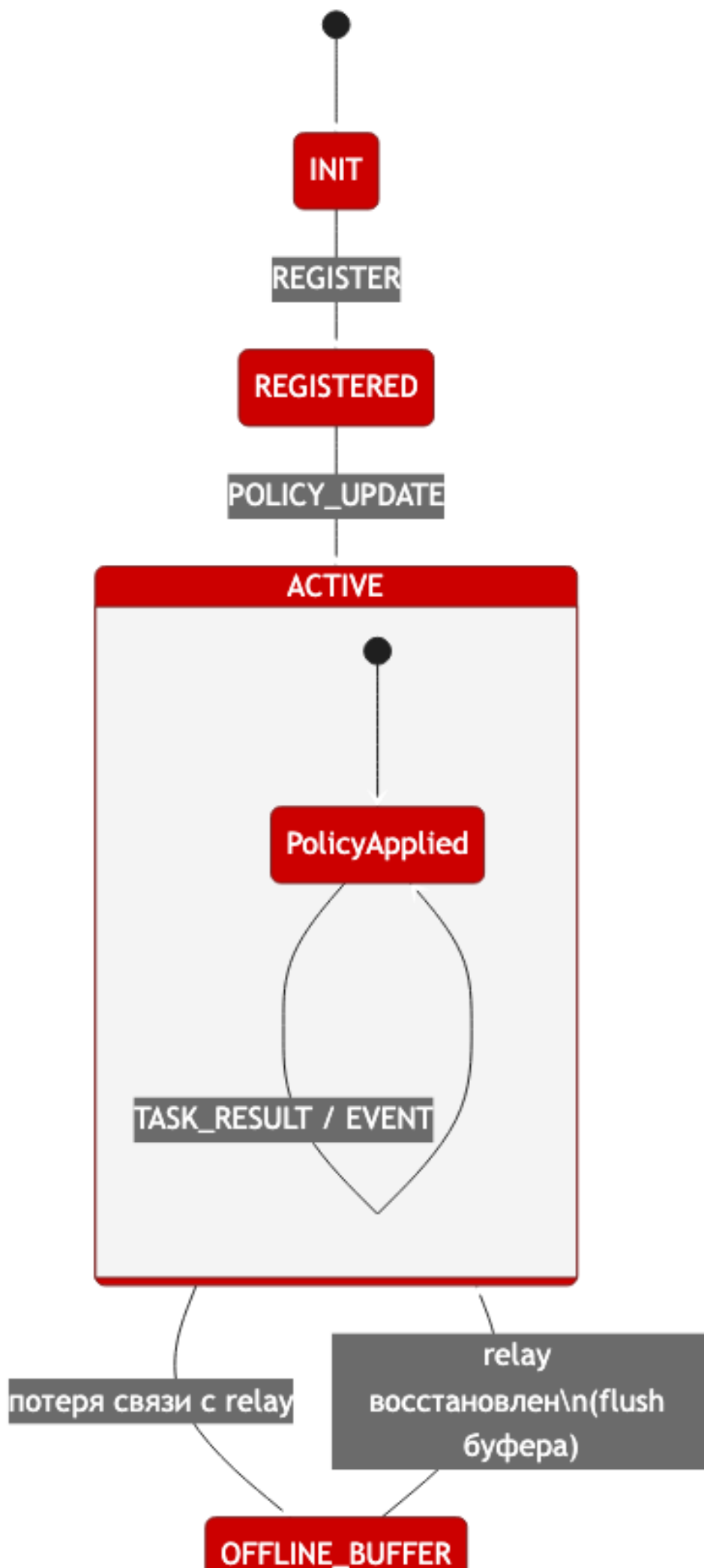


Рис. 7.1: Последовательность сообщений при экзаменационном сценарии

7.4 Схема состояний агента



8. Порядок работы с системой

8.1 Аутентификация операторов

Для доступа к системе используется двухфакторная аутентификация:

1. Ввод логина и пароля (минимум 8 символов, не менее 3 из 4 категорий символов, отсутствие в списке распространённых паролей).
2. Ввод TOTP-кода (6 цифр, обновляется каждые 30 секунд; совместимые приложения: Google Authenticator, Яндекс.Ключ).
3. Подтверждение цифровой подписью с USB-носителя (PKCS#11-токен или файл сертификата).

При бездействии оператора более 2–3 минут сессия блокируется. Возобновление — повторный ввод TOTP-кода или подключение USB-носителя.

Доступ к системе предоставляется исключительно из подсети Департамента образования и науки города Москвы. Попытки доступа из внешних сетей отклоняются на уровне межсетевого экрана.

8.2 Регламент работы в штатном режиме

В течение учебного дня ответственный за корпус выполняет следующие операции:

- Проверка панели состояния машин в начале дня. Offline-машины фиксируются; при необходимости инициируется техническое обслуживание.
- Применение учебного профиля политики к зданию или отдельным аудиториям согласно расписанию (при отключённом автоматическом планировщике).
- Мониторинг событий безопасности. Реагирование на события высокого приоритета (USB, P2P, Tamper).
- По завершении учебного дня — переключение в свободный профиль или завершение сессий.

8.3 Регламент работы в экзаменационном режиме

8.3.1 За 15 минут до начала экзамена

- Применение экзаменационного профиля политики к аудиториям, задействованным в экзамене.
- Проверка статуса всех машин в аудиториях. Все машины должны иметь статус online и активный экзаменационный профиль.
- Активация мониторинга P2P-трафика и USB-событий.

8.3.2 В ходе экзамена

- Непрерывный мониторинг ленты событий.
- При обнаружении события USB_CONNECTED (severity: high) или P2P_DETECTED — незамедлительное уведомление организатора в аудитории и блокировка устройства на котором было обнаружено событие.
- Просмотр скриншотов конкретной машины по запросу организатора.

8.3.3 По завершении экзамена

- Снятие экзаменационного профиля.
- Формирование отчёта об инцидентах за период экзамена.
- Передача отчёта администратору ИБ.

9. Информационная безопасность

9.1 Сетевая безопасность

- Доступ к API центрального сервера ограничен подсетью Департамента образования и науки города Москвы на уровне межсетевого экрана.
- Взаимодействие relay node — центральный сервер осуществляется по протоколу HTTPS/WSS с взаимной аутентификацией (mTLS). Каждый relay node имеет уникальный сертификат, выданный внутренним удостоверяющим центром.
- Взаимодействие агент — relay node внутри здания осуществляется по протоколу WebSocket. Трафик внутри здания не шифруется (локальная сеть), однако агент аутентифицируется по session_token, полученному при регистрации.
- Ограничение частоты запросов (rate limiting): 400 запросов/минуту для запросов из подсети школ; 120 запросов/минуту для прочих адресов.

9.2 Аудит и журналирование

Все действия операторов в системе фиксируются в журнале аудита. Записи журнала не могут быть удалены или изменены средствами пользовательского интерфейса. Структура записи:

```
{
  "log_id":      "uuid",
  "timestamp":   "ISO8601",
  "operator_id": "uuid",
  "operator_role": "string",
  "action":      "string",      // тип действия
  "target":      { ... },      // объект воздействия
  "payload_hash": "sha256",     // хэш параметров команды
  "ip_address":  "string",
  "session_id":  "uuid"
}
```

9.3 Защита данных учащихся

Персональные данные учащихся (ФИО, класс, место) хранятся на центральном сервере в зашифрованном виде (шифрование на уровне столбцов PostgreSQL). Скриншоты хранятся не более 30 дней в штатном режиме и не более 90 дней при наличии открытого инцидента.

Агент не сохраняет содержимое ввода пользователя (нажатия клавиш, содержимое

буфера обмена). Аналитика DNS-запросов ведётся на доменном уровне без сохранения URL или содержимого HTTP-сессий.

Соответствие требованиям Федерального закона № 152-ФЗ «О персональных данных» обеспечивается следующими мерами:

- минимизация собираемых данных;
- ограничение доступа по ролевой модели;
- автоматическое удаление по истечении срока хранения;
- ведение журнала всех операций с персональными данными.

10. Система плагинов УКК

10.1 Проблемы существующей системы плагинов Navos

Система плагинов Navos основана исключительно на Python через встроенный интерпретатор CPython. Схема работы представлена на рисунке 11.1.

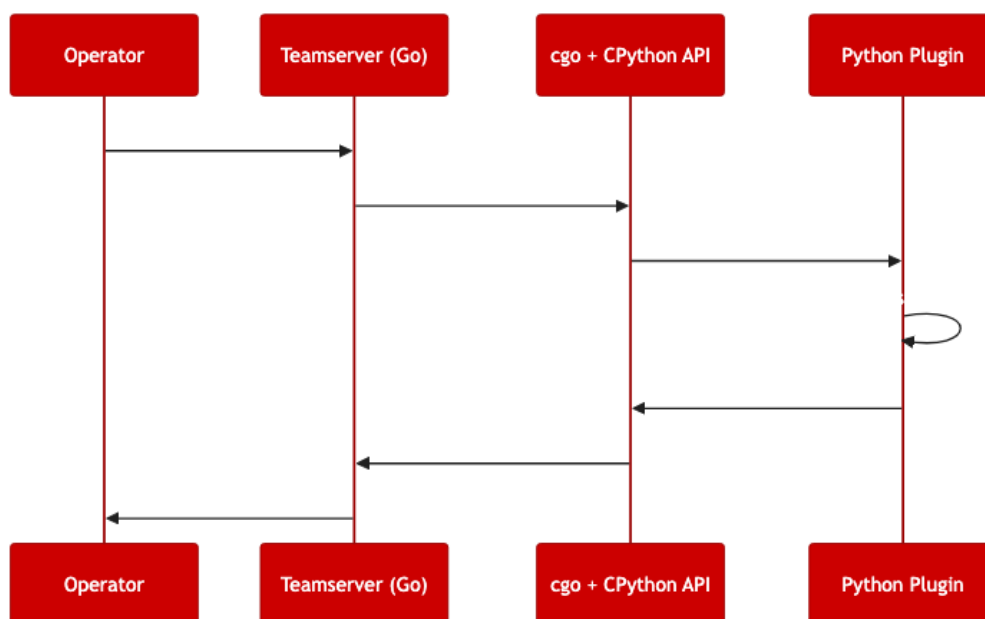


Рис. 10.1: Схема работы системы плагинов Navos

Проблемы данного подхода для УКК:

- **Только Python.** Отсутствует поддержка нативных плагинов. Для использования C++ логики требуется Python-обёртка, вызывающая shared library — три уровня косвенности.
- **Отсутствие изоляции плагинов.** Падение Python-плагина приводит к падению всего teamserver. Нет песочницы, нет локализации ошибок.
- **Отсутствие типобезопасности.** Граница Python ↔ Go строково-типизирована. Плагины передают словари без валидации схемы.
- **Отсутствие горячей перезагрузки.** Плагины загружаются при старте. Изменение плагина требует перезапуска teamserver.
- **Отсутствие декларации возможностей.** Плагины не объявляют свои требования — они вызывают любые доступные Go-функции. Нет модели разрешений, нет графа зависимостей.
- **Однопоточное выполнение.** CPython GIL означает, что вся логика плагинов выполняется в одном потоке.

10.2 Преимущества подхода Adaptix

Модель расширений Adaptix использует JSON-манифест + скомпилированную shared library:

```
{
  "name": "screenshot",
  "version": "1.0",
  "commands": [
    {
      "name": "screenshot",
      "description": "Capture screen",
      "args": [{"name": "quality", "type": "int", "default": 80}]
    }
  ]
}
```

Shared library экспортирует фиксированный C ABI, который teamserver загружает через `dlopen()`. Преимущества:

- Языконезависимость (работает всё, что может экспортировать C ABI)
- Отсутствие накладных расходов интерпретатора
- Чёткий контракт интерфейса через манифест

Однако система Adaptix имеет слабости: отсутствие compile-time типобезопасности, отсутствие template-based композиции возможностей, отсутствие гарантий стабильности ABI и отсутствие hot-swap.

10.3 Архитектура системы плагинов УКК

Основная идея — закодировать максимум контракта плагина **на этапе компиляции**, используя шаблоны для устранения целых категорий runtime-ошибок.

10.3.1 Уровень 1: ABI-граница (C, стабильный навсегда)

Внешний уровень — чистый C без манглинга имён C++, без исключений, без vtables на границе.

```
#ifdef __cplusplus
extern "C" {
#endif

#define UKK_PLUGIN_ABI_VERSION 1

typedef struct {
    uint32_t abi_version;
    uint32_t plugin_version;
```



```

    const char* name;
    const char* description;
} UKK_PluginMeta;

typedef struct {
    int (*on_load) (void* ctx);
    void (*on_unload) (void* ctx);
    int (*dispatch) (void* ctx, const uint8_t* payload,
                    uint32_t len, uint8_t** out, uint32_t* out_len);

    void* server_vtable;
} UKK_PluginInterface;

const UKK_PluginInterface* ukk_plugin_entry(void);

#ifdef __cplusplus
}
#endif

```

Этот C ABI — единственный контракт, который должен оставаться стабильным. Всё выше него может свободно эволюционировать.

10.3.2 Уровень 2: C++ Plugin SDK

Авторы плагинов никогда не работают с C ABI напрямую. Они наследуются от CRTP-базы, которая автоматически генерирует ABI-связку.

```

// plugin_sdk.hpp

struct Cap_Screenshot    {};
struct Cap_NetPolicy     {};
struct Cap_ProcessKill   {};
struct Cap_ScreenControl {};
struct Cap_AgentUpdate   {};

template<typename Derived, typename... Capabilities>
class PluginBase {
public:
    using capability_list = std::tuple<Capabilities...>;

    template<typename Cap>
    static constexpr bool has_capability() {
        return (std::is_same_v<Cap, Capabilities> || ...);
    }
}

```

```

static const UKK_PluginInterface* get_interface() {
    static UKK_PluginInterface iface = {
        .on_load    = &Derived::_abi_on_load,
        .on_unload  = &Derived::_abi_on_unload,
        .dispatch   = &Derived::_abi_dispatch,
        .server_vtable = nullptr
    };
    return &iface;
}
};

```

10.3.3 Уровень 3: Реестр команд — ядро метапрограммирования

Вместо runtime-мар строка → обработчик строится **compile-time** таблица диспетчизации.

```

template<typename Tag, typename ArgStruct, typename ResultStruct>
struct CommandDescriptor {
    using tag      = Tag;
    using args     = ArgStruct;
    using result   = ResultStruct;
    static constexpr std::string_view name = Tag::command_name;
};

template<typename... Commands>
class CommandRegistry {
public:
    using command_tuple = std::tuple<Commands...>;
    static constexpr size_t count = sizeof...(Commands);

    template<typename Tag>
    static constexpr bool has_command() {
        return (std::is_same_v<typename Commands::tag, Tag> || ...);
    }

    template<typename HandlerTable>
    static constexpr auto make_dispatch_table(HandlerTable& h) {
        return std::array{
            std::pair{Commands::name,
                    &HandlerTable::template handle<typename Commands::tag>}
            ...
        };
    }
};

```

10.3.4 Уровень 4: Загрузчик плагинов с Hot-Swap

Ключевое дополнение над Havoc и Adaptix: **версионированный hot-swap без разрыва соединений**.

```
class PluginLoader {
public:
    struct LoadedPlugin {
        void*                dl_handle;
        UKK_PluginInterface* iface;
        UKK_PluginMeta*      meta;
        std::atomic<uint32_t> active_calls{0};
        std::atomic<bool>     pending_unload{false};
    };

    std::expected<void, LoadError>
    hot_swap(PluginID id, std::filesystem::path new_so) {
        auto new_id = load(new_so);
        if (!new_id) return std::unexpected(new_id.error());

        auto& old = plugins_.at(id);
        old.pending_unload.store(true);

        while (old.active_calls.load() > 0)
            std::this_thread::yield();

        old.iface->on_unload(old.ctx);
        dlclose(old.dl_handle);
        plugins_.erase(id);
        return {};
    }
};
```

10.3.5 Уровень 5: Верификация возможностей при загрузке

При загрузке плагина движок проверяет, что плагин имеет объявленные возможности и что эти возможности разрешены текущей политикой сервера.

```
template<typename... RequiredCaps>
class CapabilityVerifier {
public:
    template<typename... PluginCaps>
    static bool verify(const ServerPolicy& policy) {
        bool has_all = (has_cap<RequiredCaps, PluginCaps...>() && ...);
        if (!has_all) return false;
    }
};
```

```
        bool permitted = (policy.permits<RequiredCaps>() && ...);  
        return permitted;  
    }  
};
```

10.3.6 Уровень 6: Compile-time сериализация через Boost.PFR

Zero-boilerplate сериализация структур аргументов/результатов с использованием Boost.PFR (неинтрузивная рефлексия структур):

```
template<typename T>  
std::vector<uint8_t> serialize(const T& obj) {  
    msgpack::sbuffer buf;  
    msgpack::packer<msgpack::sbuffer> pk(buf);  
  
    boost::pfr::for_each_field(obj, [&](const auto& field) {  
        pk.pack(field);  
    });  
  
    return {buf.data(), buf.data() + buf.size()};  
}
```

Авторы плагинов определяют простые структуры и получают сериализацию бесплатно — без схем, без кодогенерации.

11. Инфраструктура и развёртывание

11.1 Серверная инфраструктура

Вся инфраструктура системы размещается на серверах образовательного учреждения. Использование сторонних облачных провайдеров не предусмотрено.

Компонент	Требования
Центральный сервер	Python 3.10+, PostgreSQL 14+, Redis 6+, минимум 8 ГБ RAM, 100 ГБ SSD
Relay node (на здание)	Linux (чМОС или совместимый), 2 ГБ RAM, 20 ГБ HDD, канал 100 Мбит/с
Агент УКК	чМОС (Linux), X11, Python 3.8+ или скомпилированный бинарный файл, 50 МБ дискового пространства

11.2 Развёртывание агента

Агент устанавливается на рабочие станции через скрипт `install.sh`, доступный в репозитории `mos-controller`. Скрипт выполняет следующие действия:

1. Копирование бинарного файла агента и файла конфигурации в `/opt/ukk/`.
2. Создание системного пользователя `ukk-agent` с ограниченными правами.
3. Регистрация `systemd unit` для агента и `watchdog`.
4. Первичная регистрация агента на `relay node` здания.
5. Применение первоначальной политики.

```
sudo chmod +x ./install.sh
sudo ./install.sh \
  --relay-url ws://relay.building1.school.local:8765 \
  --building-id building1 \
  --room-id room-301 \
  --seat-id 1A
```

11.3 Централизованное обновление агентов

Обновление производится через API центрального сервера. Оператор загружает новый бинарный файл агента, указывает SHA-256 хэш и целевые машины. Relay node раздаёт файл агентам. Статус обновления каждой машины отображается в реальном времени.

При ошибке обновления на более чем 10% машин здания relay node приостанавливает распространение до вмешательства оператора.